PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR SEDE MANABÍ

ASIGNATURA
BASE DE DATOS EN LA NUBE

TRABAJO /INVESTIGACION
PRÁCTICA DE CLEVER CLOUD

INTEGRANTES/ALUMNO
YAKOV DAVID SENI BARCIA

DOCENTE
ALEX FERNANDO RICAURTE SEGOVIA

FECHA DE ENTREGA
Portoviejo, Octubre 2026

1. Objetivo

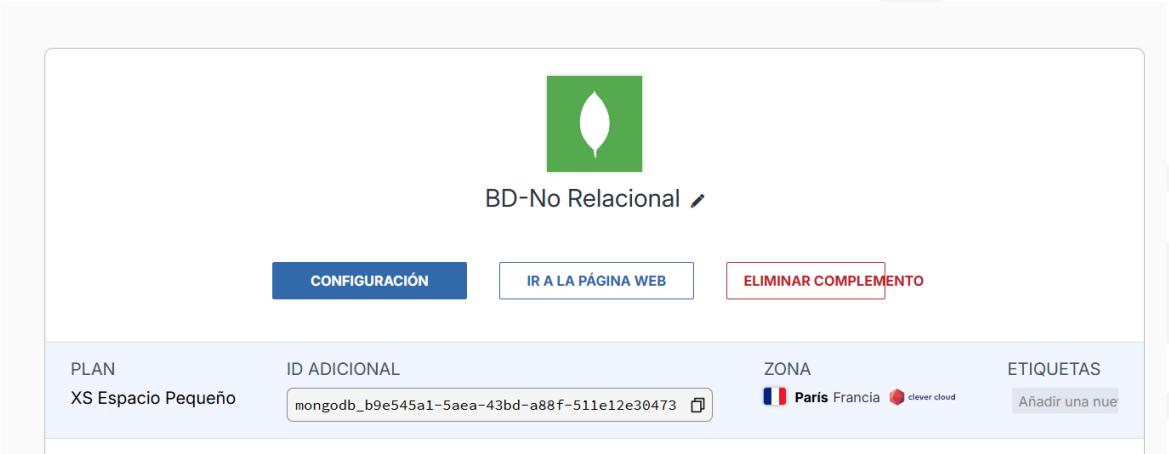
Conectar y operar con una base de datos NoSQL MongoDB alojada en Clever Cloud, utilizando una terminal de Windows y una aplicación desarrollada en Python, para realizar operaciones de inserción, consulta, actualización y eliminación de documentos.

2. Desarrollo de la práctica

2.1. Configuración de MongoDB en Clever Cloud

Se utilizó la plataforma Clever Cloud para disponer de una instancia de MongoDB en la nube. Se identificaron los parámetros necesarios para establecer una conexión remota, como el host, el puerto, el usuario y el nombre de la base de datos.

Figura 1. Configuración de la instancia MongoDB en Clever Cloud.



2.2. Conexión remota mediante MongoDB Shell

Se realizó la conexión desde PowerShell en Windows utilizando la consola MongoDB Shell clásica, versión 4.4.31, compatible con la instancia MongoDB 4.0.3 disponible en Clever Cloud.

Para verificar la conexión se ejecutó el comando db.runCommand({ ping: 1 }), obteniendo como resultado { "ok": 1 }, lo que confirmó la comunicación con el servidor remoto.

Figura 2. Conexión exitosa a MongoDB desde PowerShell.

```
PS C:\Users\exodu> & "C:\Program Files\MongoDB\Server\4.4\bin\mongo.exe" "mongodb://bghmp54idula05rg6zya-mongodb.services.clever-cloud.com:2257/bghmp54idula
05rg6zya" --username "ufi8cvjnyec0yqqtwdtm" --authenticationDatabase "bghmp54idula05rg6zya" --password
MongoDB shell version v4.4.31
Enter password:

connecting to: mongodb://bghmp54idula05rg6zya-mongodb.services.clever-cloud.com:2257/bghmp54idula05rg6zya?authSource=bghmp54idula05rg6zya&compressors=disabl
ed&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("79d952ad-41ba-4d7f-a677-0166240b7f80") }
MongoDB server version: 4.0.3
WARNING: shell and server versions do not match
Welcome to the MongoDB shell.
For interactive help, type "help".
For more comprehensive documentation, see
        https://docs.mongodb.com/
Questions? Try the MongoDB Developer Community Forums
        https://community.mongodb.com
---
The server generated these startup warnings when booting:
2026-10-13T16:17:18.539+0000 I CONTROL  [initandlisten]
2026-10-13T16:17:18.539+0000 I CONTROL  [initandlisten] ** WARNING: /proc/sys/vm/overcommit_memory is 2
2026-10-13T16:17:18.539+0000 I CONTROL  [initandlisten] **          Journaling works best with it set to 0 or 1
2026-10-13T16:17:18.539+0000 I CONTROL  [initandlisten]
2026-10-13T16:17:18.539+0000 I CONTROL  [initandlisten] ** WARNING: /sys/kernel/mm/transparent_hugepage/enabled is 'always'.
2026-10-13T16:17:18.539+0000 I CONTROL  [initandlisten] **        We suggest setting it to 'never'
2026-10-13T16:17:18.539+0000 I CONTROL  [initandlisten]
---
> db.runCommand({ ping: 1 })
{ "ok" : 1 }
>
```

2.3. Inserción y consulta de documentos

Se utilizó el método insertMany() para insertar dos documentos en la colección estudiantes, correspondientes a Laura Martínez y Pedro Sánchez.

Posteriormente se ejecutó find().pretty() para consultar y visualizar los documentos almacenados.

Figura 3. Inserción y consulta de estudiantes en MongoDB.

```
> db.estudiantes.insertMany([
...     {
...       nombre: "Laura Martínez",
...       email: "laura@email.com",
...       edad: 22,
...       carrera: "Psicología",
...       materias: ["Psicología Clínica", "Neuropsicología"],
...       activo: true
...     },
...     {
...       nombre: "Pedro Sánchez",
...       email: "pedro@email.com",
...       edad: 24,
...       carrera: "Administración",
...       materias: ["Contabilidad", "Finanzas"],
...       activo: true
...     }
... ])
{
        "acknowledged" : true,
        "insertedIds" : [
                ObjectId("6ace61f4c6ef4c2bc73843dd"),
                ObjectId("6ace61f4c6ef4c2bc73843de")
        ]
}
> db.estudiantes.find().pretty()
{
        "_id" : ObjectId("6ace61f4c6ef4c2bc73843dd"),
        "nombre" : "Laura Martínez",
        "email" : "laura@email.com",
        "edad" : 22,
        "carrera" : "Psicología",
        "materias" : [
                "Psicología Clínica",
                "Neuropsicología"
        ],
        "activo" : true
}
```

2.4. Actualización de documentos

Se utilizó el método updateOne() para modificar la edad de Laura Martínez de 22 a 23 años.

El resultado mostró modifiedCount: 1, confirmando que el documento fue actualizado correctamente. Posteriormente, se realizó una consulta para verificar el cambio.

Figura 4. Actualización de los datos de un estudiante.

```
> db.estudiantes.updateOne(
...   { nombre: "Laura Martínez" },
...   { $set: { edad: 23 } }
... )
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
> db.estudiantes.find({ nombre: "Laura Martínez" }).pretty()
{
        "_id" : ObjectId("6ace61f4c6ef4c2bc73843dd"),
        "nombre" : "Laura Martínez",
        "email" : "laura@email.com",
        "edad" : 23,
        "carrera" : "Psicología",
        "materias" : [
                "Psicología Clínica",
                "Neuropsicología"
        ],
        "activo" : true
}
```

2.5. Eliminación de documentos

Se ejecutó el método deleteOne() para eliminar el documento correspondiente a Pedro Sánchez.

El resultado deletedCount: 1 confirmó la eliminación. La consulta posterior permitió verificar que Laura Martínez permanecía registrada en la colección.

Figura 5. Eliminación de un estudiante y verificación de los registros.

```
> db.estudiantes.deleteOne({ nombre: "Pedro Sánchez" })
{ "acknowledged" : true, "deletedCount" : 1 }
> db.estudiantes.find().pretty()
{
        "_id" : ObjectId("6ace61f4c6ef4c2bc73843dd"),
        "nombre" : "Laura Martínez",
        "email" : "laura@email.com",
        "edad" : 23,
        "carrera" : "Psicología",
        "materias" : [
                "Psicología Clínica",
                "Neuropsicología"
        ],
        "activo" : true
}
```

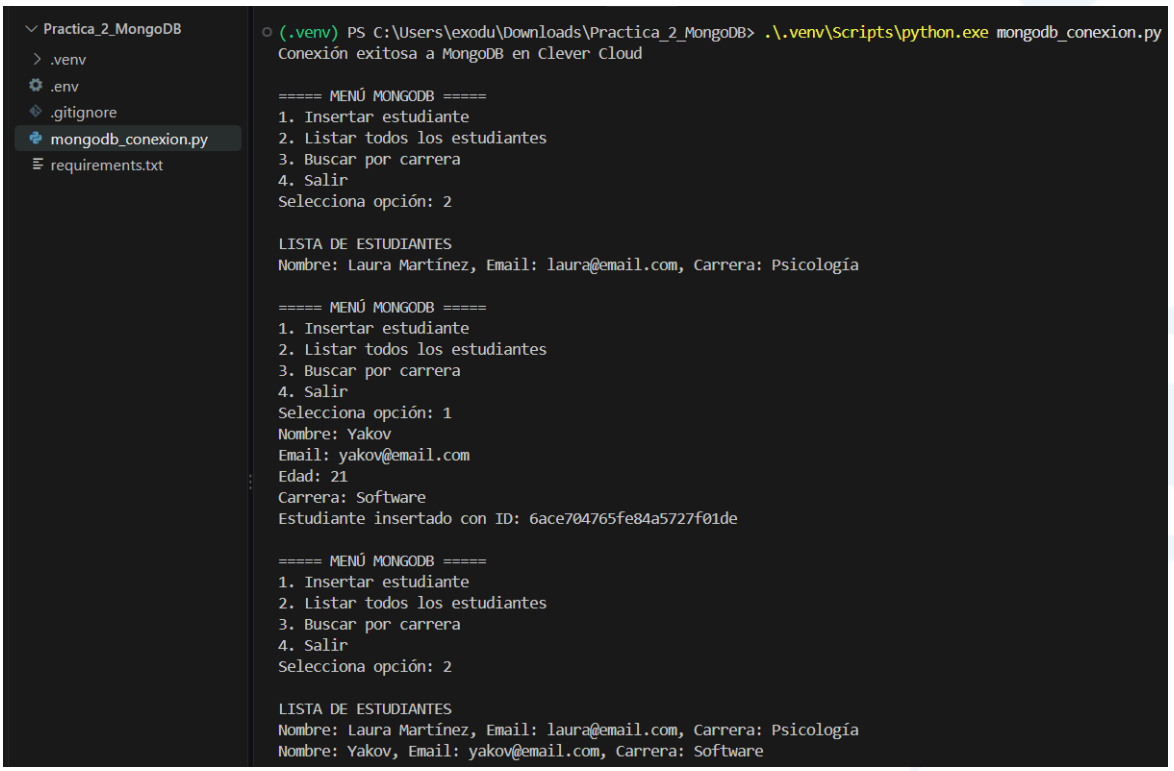
2.6. Desarrollo de la aplicación Python

Se desarrolló una aplicación en Visual Studio Code utilizando Python y la biblioteca PyMongo para establecer la conexión con MongoDB en Clever Cloud.

El programa dispone de un menú con opciones para insertar estudiantes, listar registros, buscar estudiantes por carrera y salir.

Durante las pruebas se verificó la conexión exitosa, se consultó la información almacenada y se insertó un nuevo estudiante, llamado Yakov, de 21 años, perteneciente a la carrera Software.

Figura 6. Aplicación Python conectada a MongoDB, con operaciones de inserción y listado.



3. Comparación entre MongoDB y MySQL

Característica	MongoDB	MySQL
Modelo de datos	NoSQL orientado a documentos	Relacional
Almacenamiento	Colecciones y documentos BSON	Tablas, filas y columnas
Estructura	Esquema flexible	Esquema definido
Lenguaje de consulta	Métodos y operadores de MongoDB	SQL

Relaciones	Documentos embebidos y referencias	Claves primarias y foráneas
Aplicaciones	Datos semiestructurados y modelos flexibles	Aplicaciones con datos estructurados y relaciones definidas

MongoDB permite almacenar documentos con estructuras flexibles, mientras que MySQL organiza la información mediante tablas relacionadas. Ambos sistemas ofrecen mecanismos para consultar, administrar y mantener información, pero emplean diferentes modelos de almacenamiento.

4. Conclusiones

Durante el desarrollo de la práctica se logró establecer una conexión remota con MongoDB alojado en Clever Cloud, utilizando PowerShell y MongoDB Shell.

Se realizaron correctamente operaciones CRUD, incluyendo inserción, consulta, actualización y eliminación de documentos. Además, se desarrolló una aplicación en Python que permitió conectarse a la base de datos y gestionar registros de estudiantes mediante un menú interactivo.

Finalmente, se identificaron las principales diferencias entre MongoDB y MySQL, comprendiendo la importancia de seleccionar un sistema de base de datos según las características de la aplicación y la estructura de sus datos.